

Analogue to Digital Converter

Analogue to Digital Converter, or ADC, is a data converter which allows digital circuits to interface with the real world by encoding an analogue signal into a binary code

Analogue-to-Digital Converters, (ADCs) allow micro-processor controlled circuits, Arduinos, Raspberry Pi, and other such digital logic circuits to communicate with the real world. In the real world, analogue signals have continuously changing values which come from various sources and sensors which can measure sound, light, temperature or movement, and many digital systems interact with their environment by measuring the analogue signals from such transducers.

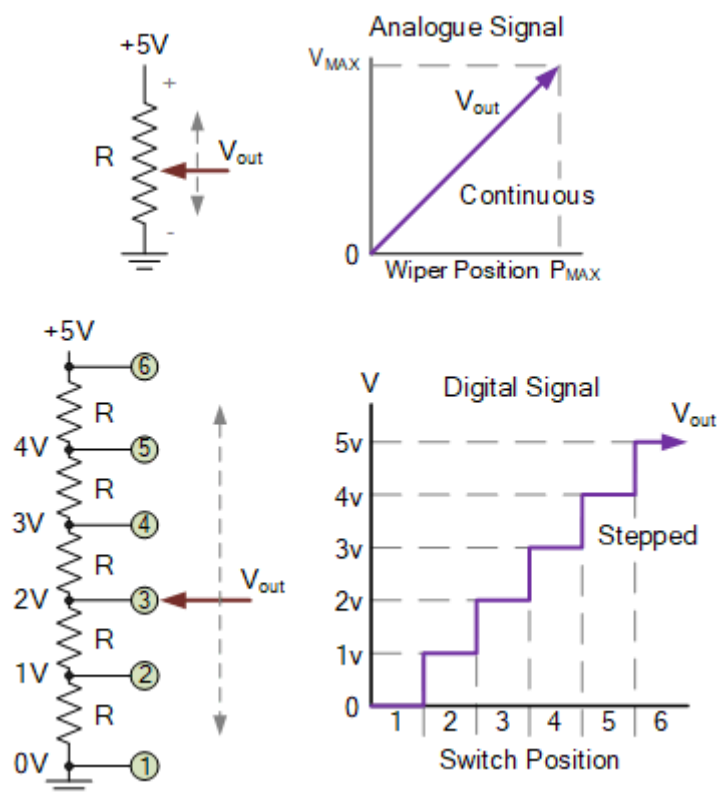
While analogue signals can be continuous and provide an infinite number different voltage values, digital circuits on the other hand work with binary signal which have only two discrete states, a logic "1" (HIGH) or a logic "0" (LOW). So it is necessary to have an electronic circuit which can convert between the two different domains of continuously changing analogue signals and discrete digital signals, and this is where *Analogue-to-Digital Converters* (A/D) come in.

Basically an analogue to digital converter takes a snapshot of an analogue voltage at one instant in time and produces a digital output code which represents this analogue voltage. The number of binary digits, or bits used to represent this analogue voltage value depends on the resolution of an A/D converter.

For example a 4-bit ADC will have a resolution of one part in 15, ($2^4 - 1$) whereas an 8-bit ADC will have a resolution of one part in 255, ($2^8 - 1$). Thus an analogue to digital converter takes an unknown continuous analogue signal and converts it into an "n"- bit binary number of 2^n bits.

But first let us remind ourselves of the differences between an analogue (or analog) signal and a digital signal as shown:

Analogue and Digital Signals



Here we can see that as the wiper terminal of the potentiometer is rotated between between 0 volts and V_{MAX} , it produces a continuous output signal (or voltage) which has an infinite number of output values relative to the wiper position. As the potentiometers wiper is adjusted from one position to the next, there is no sudden or step change between the two voltage levels thereby producing a continuously variable output voltage. Examples of analogue signals include temperature, pressure, liquid levels and light intensity.

For a digital circuit the potentiometer wiper has been replaced by a single rotary switch which is connected in turn to each junction of the series resistor chain, forming a basic potential divider network. As the switch is rotated from one position (or node) to the next the output voltage, V_{OUT} changes quickly in discrete and distinctive voltage steps representing multiples of 1.0 volts on each switching action or step as shown.

So for example, the output voltage will be 2 volts, 3 volts, 5 volts, etc. but NOT 2.5V, 3.1V or 4.6V. Finer output voltage levels could easily be produced by using a multi-positional switch and increasing the number of resistive elements within the potential divider network, therefore increasing the number of discrete switching steps.

Then we can see that the major differences between an analogue signal and a digital signal is that an “Analogue” quantity is continuously changing over time while a “Digital” quantity has discrete (step by step) values. “LOW” to “HIGH” or “HIGH” to “LOW”.

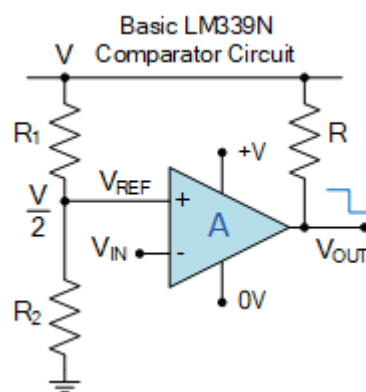
So how can we convert a continuously changing signal with an infinite number of values to one which has distinct values or steps for use by a digital circuit.

Analogue-to-Digital Converter

The process of taking an analogue voltage signal and converting it into an equivalent digital signal can be done in many different ways, and while there are many analogue-to-digital converter chips such as the ADC08xx series available from various manufacturers, it is possible to build a simple ADC using discrete components. One simple and easy way is by using parallel encoding, also known as *flash*, *simultaneous*, or *multiple comparator* converters in which comparators are used to detect different voltage levels and output their switching state to an encoder.

Parallel or “Flash” A/D converters use a series of interconnected but equally spaced comparators and voltage references generated by a series network of precision resistors for generating an equivalent output code for a particular n-bit resolution. The advantage of parallel or flash converters is that they are simple to construct and do not require any timing clocks as the instant an analogue voltage is applied to the comparator inputs, it is compared against a reference voltage. Consider the comparator circuit below.

Comparator Circuit



An analogue comparator such as the LM339N which has two analogue inputs, one positive and one negative, and which can be used to compare the magnitudes of two different voltage levels. A voltage input, (V_{IN}) signal is applied to one input of the comparator, while a reference voltage, (V_{REF}) to the other. A comparison of the two voltage levels at the comparator's input is made to determine the comparators digital logic output state, either a “1” or a “0”.

The reference voltage, V_{REF} is compared against the input voltage, V_{IN} applied to the other input. For an LM339 comparator, if the input voltage is less than the reference voltage, ($V_{IN} < V_{REF}$) the output is “OFF”, and if it is greater than the reference voltage, ($V_{IN} > V_{REF}$) the output will be “ON”. Thus a comparator compares two voltage levels and determines which one of the two is higher.

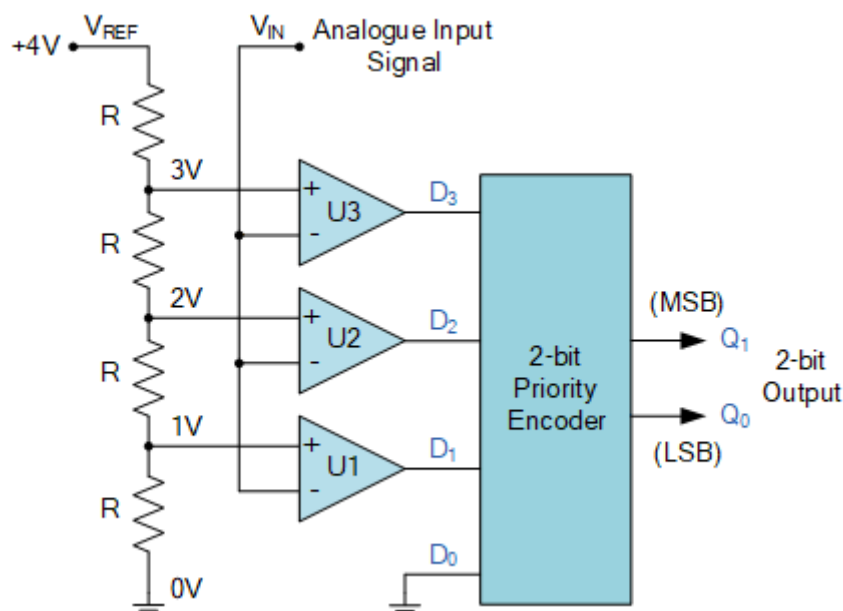
In our simple example above, V_{REF} is obtained from the voltage divider network setup by R_1 and R_2 . If the two resistors are of equal values, that is $R_1 = R_2$, then clearly the reference voltage level will be equal to half the supply voltage, or $V/2$. So for a comparator with an open-collector output, if V_{IN} is less than $V/2$, the output is HIGH, and if V_{IN} is greater than $V/2$, the output is LOW acting as a 1-bit ADC.

But by adding more resistors to the voltage divider network we can effectively “divide” the supply voltage by an amount determined by the resistances of the resistors. However, the more resistors we use in the voltage divider network the more comparators will be required.

In general, $2^n - 1$ comparators would be required for conversion of an “n”-bit binary output, where “n” is typically in the range from 8 to 16. In our example above, the single bit ADC used $2^1 - 1$, which equals “1” comparator to determine if V_{IN} was greater or smaller than the $V/2$ reference voltage.

If we now create a 2-bit ADC, then we will need $2^2 - 1$ which is “3” comparators as we need four different voltage levels corresponding to the 4 digital values required for a 4-to-2 bit encoder circuit as shown.

2-bit Analogue to Digital Converter Circuit



This will give us a 2-bit output code for all four possible values of analogue input of:

2-bit A/D converter Output

Analogue Voltage (V_{IN})	Comparator Outputs				Digital Outputs	
	D_3	D_2	D_1	D_0	Q_1	Q_0
0 to 1 V	0	0	0	0	0	0
1 to 2 V	0	0	1	X	0	1
2 to 3 V	0	1	X	X	1	0
3 to 4 V	1	X	X	X	1	1

Where: "X" is a "don't care", that is either a logic "0" or a logic "1" condition.

So how does this *analogue-to-digital converter* work. For an A/D converters to be useful it has to produce a meaningful digital representation of the analogue input signal. Here in this simple 2-bit ADC example we have assumed for simplicity that the input voltage V_{IN} is between 0 and 4 volts, so have set V_{REF} and the resistive voltage-divider network to drop 1 volt across each resistor.

When V_{IN} is between 0 and 1 volt, ($<1V$) the input on all three comparators will be less than the reference voltage, so their outputs will be LOW and the encoder will output a binary zero (00) condition on pins Q_0 and Q_1 . When V_{IN} increases and exceeds 1 volt but is less than 2 volts, ($1V < V_{IN} < 2V$) comparator U1 which has a reference voltage input set at 1 volt, will detect this voltage difference and produce a HIGH output. The priority encoder which is used as the 4-to-2 bit encoding detects the change of input at D_1 and produces a binary output of "1" (01).

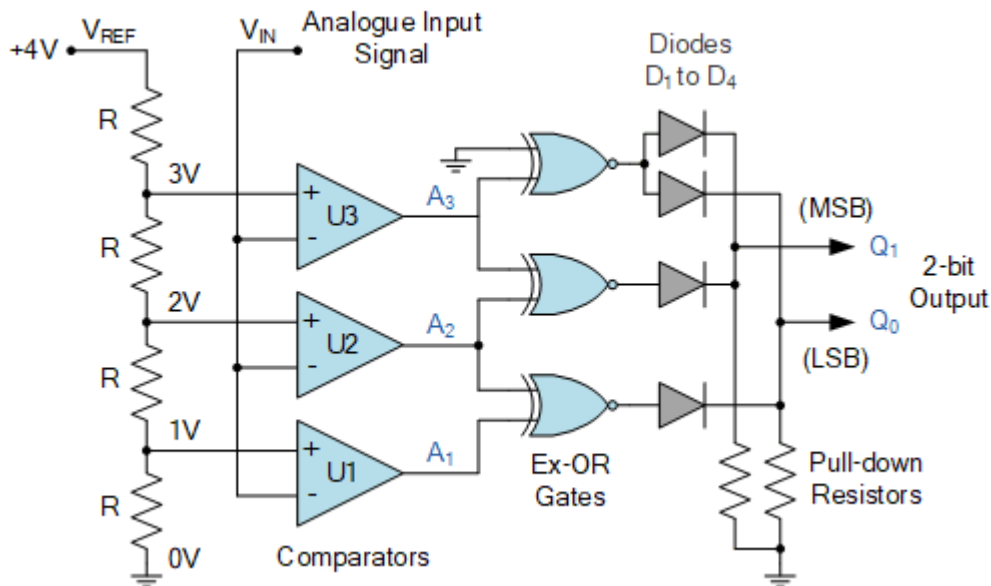
Note that a **Priority Encoder** such as the TTL 74LS148 allocates a priority level to each individual input. The priority encoders output corresponds to the currently active input which has the highest priority. So when an input with a higher priority (D_1 compared to D_0) is present, all other inputs with a lower priority will be ignored. So if there

are two or more inputs at logic level “1” at the same time, the actual output code on D_0 and D_1 would only correspond to the input with the highest designated priority.

So now as V_{IN} increases above 2 volts, the next reference voltage level, comparator U2 detects the change and produces a HIGH output. But because input D_2 has a higher priority than inputs D_0 or D_1 , the priority encoder outputs a binary “2” (10) code, and so on when V_{IN} exceeds 3 volts producing a binary code output of “3” (11). Clearly as V_{IN} reduces or changes between each reference voltage level, each comparator will output either a HIGH or a LOW condition to the encoder which in turn produces a 2-bit binary code between 00 and 11 relative to V_{IN} .

This is all well and good, but priority encoders are not available as 4-to-2 bit devices, and if we use a commercially available one such as the TTL 74LS148 or its CMOS 4532 equivalent which are both 8-bit devices, then six of the binary bits would not be used. But a simple encoder circuit can be made using digital Ex-OR gates and a matrix of signal diodes as shown.

2-bit ADC Using Diodes



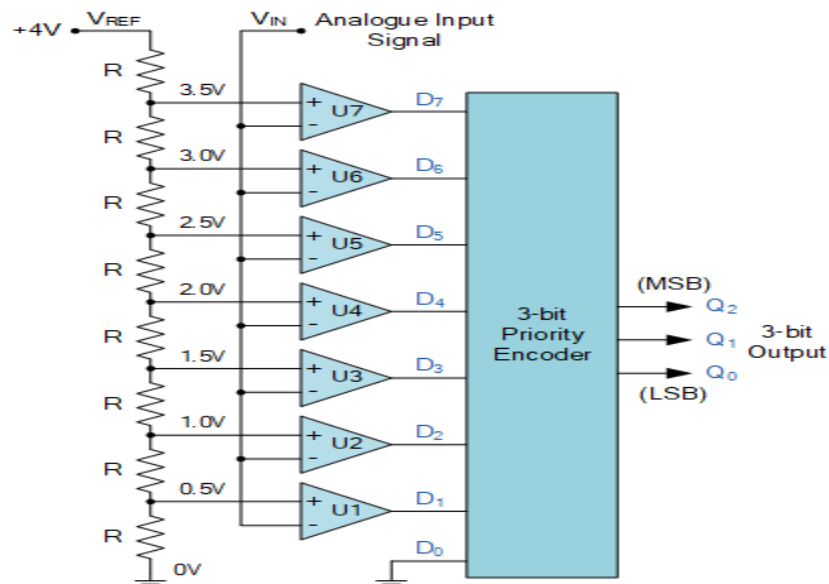
Here the outputs of the comparators are encoded using Exclusive-OR gates before being fed to the diodes. Two external pull-down resistor are used at their outputs and ground (0V) to ensure a LOW condition and stop the outputs from floating when the diodes are reverse biased. So as with the previous circuit, depending upon the value of V_{IN} determines which comparator produces an output signal HIGH (or LOW) to the exclusive-OR gates producing a HIGH output if one input or the other input is HIGH, but not both, (Boolean expression is $Q = A.B + A.B$). These Ex-OR gates could also be constructed using the combinational logic AND–OR–NAND gates.

The problem here with both designs of 4-to-2 converters is that the *resolution* of this simple 2-bit A/D converter is 1 volt because as we have seen, the analogue input voltage at V_{IN} must change by one full volt in order for the encoder to change its output code. One way to improve the resolution of the output is to increase it to a 3-bit A/D converter using more comparators.

3-bit Analogue-to-Digital Converter

The parallel ADC above converts the analogue input voltage in the range from 0 to over 3 volts to produce a 2-bit binary code. Since a 3-bit digital logic system can generate $2^3 = 8$ different digital outputs, the analogue input voltage can therefore be compared against eight reference voltage levels with each voltage level being equal to one eighth ($V/8$) of the reference voltage. Thus we could now measure a resolution of 0.5 ($4/8$) volts and would require $2^3 - 1$ comparators for a 3-bit binary code output between 000 (0) and 111 (7) as shown.

3-bit Analogue to Digital Converter Circuit



This will give us a 3-bit output code for all eight possible values of analogue input of:

3-bit A/D converter Output

[illegible]

Where again “X” is a “don’t care”, that is either a logic “0” or a logic “1” input condition.

Then we can see, that by increasing the resolution of the ADC, not only increases the number of output binary bits, but also increases the number of comparators and voltage levels needed. So a 4-bit resolution requires 15 ($2^4 - 1$) comparators, an 8-bit resolution requires 255 ($2^8 - 1$) comparators, while a 10-bit analogue-to-digital converter would require 1023 comparators, etc. So for this type of **Analogue-to-Digital Converter** circuit, the higher the number of output bits required, the more complex becomes the circuit.

However, the advantage of this type of parallel or flash A/D converter is that its real-time conversion rate is relatively fast and can be easily built as part of a project if only a few binary bits are required to produce a read out on a digital display to show the voltage value of an analogue input signal.

As well as taking an analogue input signal from a sensor or transducer and converting it using an analogue-to-digital converter into a digital binary code as part of an input interfacing circuit, we can also take a binary code and convert it into an equivalent analogue quantity using a Digital-to-Analogue Converter for output interfacing to control a motor or actuator, or commonly in audio applications.

In the next tutorial about digital circuits, we will look at the a **digital-to-analogue converter**, or simply DAC, which are the exact opposite of the *analogue-to-digital converters* looked at here. DAC’s use op-amps and resistive divider networks to convert an “n”-bit binary number into an equivalent analogue output voltage, or current signal.

Analogue-to-Digital Converters, (ADCs) allow micro-processor controlled circuits, Arduinos, Raspberry Pi, and other such digital logic circuits to communicate with the real world. In the real world, analogue signals have continuously changing values which come from various sources and sensors which can measure sound, light, temperature or movement, and many digital systems interact with their environment by measuring the analogue signals from such transducers.

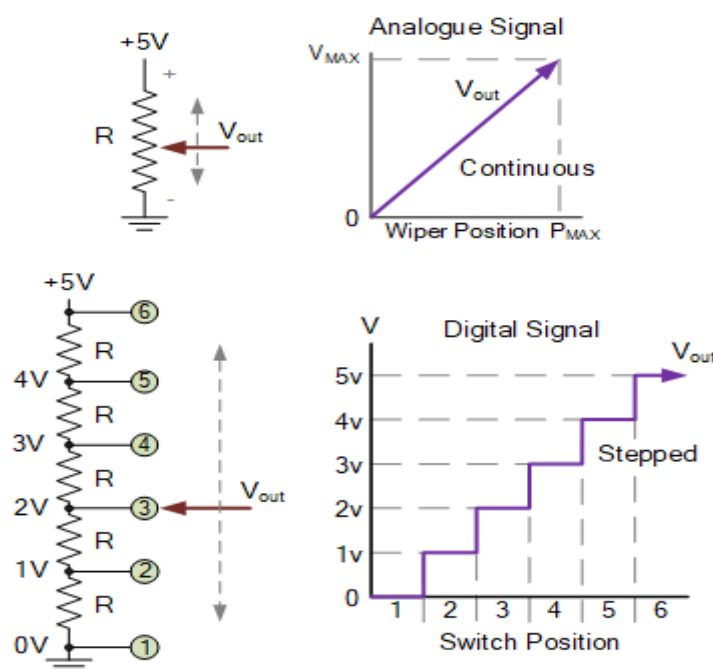
While analogue signals can be continuous and provide an infinite number different voltage values, digital circuits on the other hand work with binary signal which have only two discrete states, a logic “1” (HIGH) or a logic “0” (LOW). So it is necessary to have an electronic circuit which can convert between the two different domains of continuously changing analogue signals and discrete digital signals, and this is where *Analogue-to-Digital Converters (A/D)* come in.

Basically an analogue to digital converter takes a snapshot of an analogue voltage at one instant in time and produces a digital output code which represents this analogue voltage. The number of binary digits, or bits used to represent this analogue voltage value depends on the resolution of an A/D converter.

For example a 4-bit ADC will have a resolution of one part in 15, ($2^4 - 1$) whereas an 8-bit ADC will have a resolution of one part in 255, ($2^8 - 1$). Thus an analogue to digital converter takes an unknown continuous analogue signal and converts it into an “n”- bit binary number of 2^n bits.

But first let us remind ourselves of the differences between an analogue (or analog) signal and a digital signal as shown:

Analogue and Digital Signals



Here we can see that as the wiper terminal of the potentiometer is rotated between 0 volts and V_{MAX} , it produces a continuous output signal (or voltage) which has an infinite number of output values relative to the wiper position. As the potentiometer's wiper is adjusted from one position to the next, there is no sudden or step change between the two voltage levels thereby producing a continuously variable output voltage. Examples of analogue signals include temperature, pressure, liquid levels and light intensity.

For a digital circuit the potentiometer wiper has been replaced by a single rotary switch which is connected in turn to each junction of the series resistor chain, forming a basic potential divider network. As the switch is rotated from one position (or node) to the next the output voltage, V_{OUT} changes quickly in discrete and distinctive voltage steps representing multiples of 1.0 volts on each switching action or step as shown.

So for example, the output voltage will be 2 volts, 3 volts, 5 volts, etc. but NOT 2.5V, 3.1V or 4.6V. Finer output voltage levels could easily be produced by using a multi-positional switch and increasing the number of resistive elements within the potential divider network, therefore increasing the number of discrete switching steps.

Then we can see that the major differences between an analogue signal and a digital signal is that an "Analogue" quantity is continuously changing over time while a "Digital" quantity has discrete (step by step) values. "LOW" to "HIGH" or "HIGH" to "LOW".

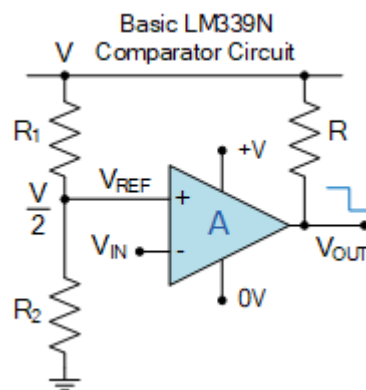
So how can we convert a continuously changing signal with an infinite number of values to one which has distinct values or steps for use by a digital circuit.

Analogue-to-Digital Converter

The process of taking an analogue voltage signal and converting it into an equivalent digital signal can be done in many different ways, and while there are many analogue-to-digital converter chips such as the ADC08xx series available from various manufacturers, it is possible to build a simple ADC using discrete components. One simple and easy way is by using parallel encoding, also known as *flash*, *simultaneous*, or *multiple comparator* converters in which comparators are used to detect different voltage levels and output their switching state to an encoder.

Parallel or "Flash" A/D converters use a series of interconnected but equally spaced comparators and voltage references generated by a series network of precision resistors for generating an equivalent output code for a particular n-bit resolution. The advantage of parallel or flash converters is that they are simple to construct and do not require any timing clocks as the instant an analogue voltage is applied to the comparator inputs, it is compared against a reference voltage. Consider the comparator circuit below.

Comparator Circuit



An analogue comparator such as the LM339N which has two analogue inputs, one positive and one negative, and which can be used to compare the magnitudes of two different voltage levels. A voltage input, (V_{IN}) signal is applied to one input of the comparator, while a reference voltage, (V_{REF}) to the other. A comparison of the two voltage levels at the comparator's input is made to determine the comparator's digital logic output state, either a "1" or a "0".

The reference voltage, V_{REF} is compared against the input voltage, V_{IN} applied to the other input. For an LM339 comparator, if the input voltage is less than the reference voltage, ($V_{IN} < V_{REF}$) the output is "OFF", and if it is greater than the reference voltage, ($V_{IN} > V_{REF}$) the output will be "ON". Thus a comparator compares two voltage levels and determines which one of the two is higher.

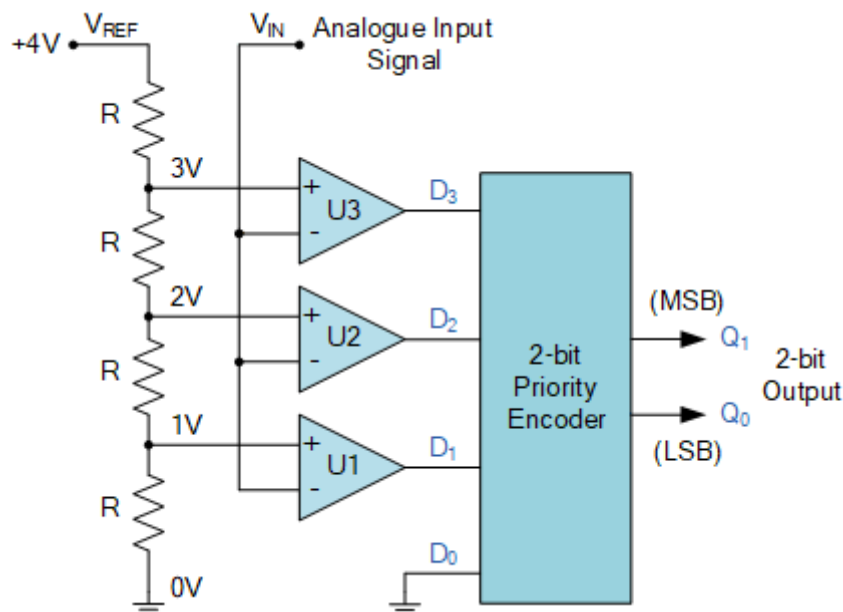
In our simple example above, V_{REF} is obtained from the voltage divider network setup by R_1 and R_2 . If the two resistors are of equal values, that is $R_1 = R_2$, then clearly the reference voltage level will be equal to half the supply voltage, or $V/2$. So for a comparator with an open-collector output, if V_{IN} is less than $V/2$, the output is HIGH, and if V_{IN} is greater than $V/2$, the output is LOW acting as a 1-bit ADC.

But by adding more resistors to the voltage divider network we can effectively "divide" the supply voltage by an amount determined by the resistances of the resistors. However, the more resistors we use in the voltage divider network the more comparators will be required.

In general, $2^n - 1$ comparators would be required for conversion of an “n”-bit binary output, where “n” is typically in the range from 8 to 16. In our example above, the single bit ADC used $2^1 - 1$, which equals “1” comparator to determine if V_{IN} was greater or smaller than the $V/2$ reference voltage.

If we now create a 2-bit ADC, then we will need $2^2 - 1$ which is “3” comparators as we need four different voltage levels corresponding to the 4 digital values required for a 4-to-2 bit encoder circuit as shown.

2-bit Analogue to Digital Converter Circuit



This will give us a 2-bit output code for all four possible values of analogue input of:

2-bit A/D converter Output

Analogue Voltage (V_{IN})	Comparator Outputs				Digital Outputs	
	D_3	D_2	D_1	D_0	Q_1	Q_0
0 to 1 V	0	0	0	0	0	0
1 to 2 V	0	0	1	X	0	1
2 to 3 V	0	1	X	X	1	0
3 to 4 V	1	X	X	X	1	1

Where: “X” is a “don’t care”, that is either a logic “0” or a logic “1” condition.

So how does this *analogue-to-digital converter* work. For an A/D converters to be useful it has to produce a meaningful digital representation of the analogue input signal. Here in this simple 2-bit ADC example we have assumed for simplicity that the input voltage V_{IN} is between 0 and 4 volts, so have set V_{REF} and the resistive voltage-divider network to drop 1 volt across each resistor.

When V_{IN} is between 0 and 1 volt, ($<1V$) the input on all three comparators will be less than the reference voltage, so their outputs will be LOW and the encoder will output a binary zero (00) condition on pins Q_0 and Q_1 . When

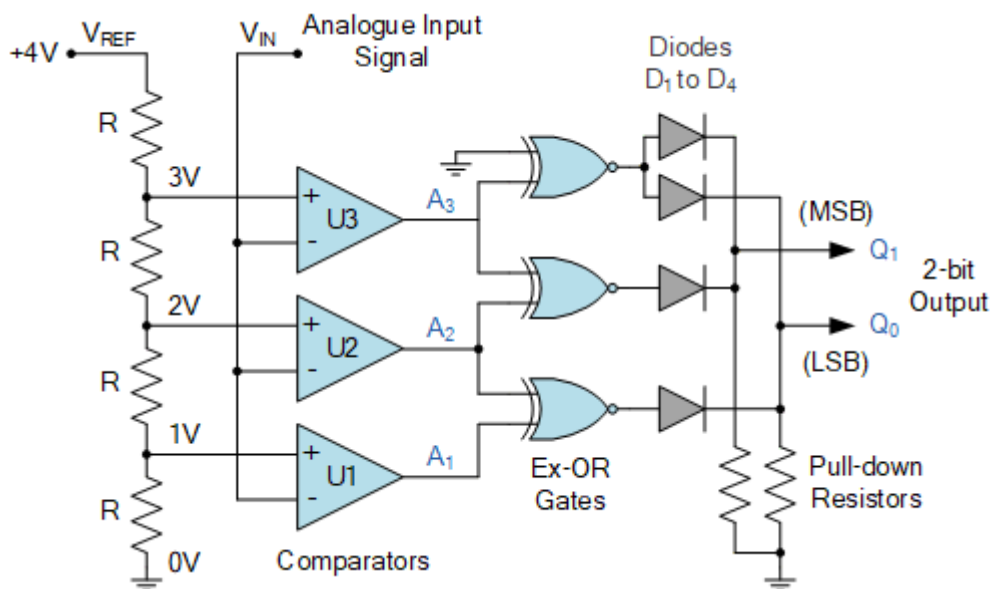
V_{IN} increases and exceeds 1 volt but is less than 2 volts, ($1V < V_{IN} < 2V$) comparator U1 which has a reference voltage input set at 1 volt, will detect this voltage difference and produce a HIGH output. The priority encoder which is used as the 4-to-2 bit encoding detects the change of input at D_1 and produces a binary output of “1” (01).

Note that a **Priority Encoder** such as the TTL 74LS148 allocates a priority level to each individual input. The priority encoders output corresponds to the currently active input which has the highest priority. So when an input with a higher priority (D_1 compared to D_0) is present, all other inputs with a lower priority will be ignored. So if there are two or more inputs at logic level “1” at the same time, the actual output code on D_0 and D_1 would only correspond to the input with the highest designated priority.

So now as V_{IN} increases above 2 volts, the next reference voltage level, comparator U2 detects the change and produces a HIGH output. But because input D_2 has a higher priority than inputs D_0 or D_1 , the priority encoder outputs a binary “2” (10) code, and so on when V_{IN} exceeds 3 volts producing a binary code output of “3” (11). Clearly as V_{IN} reduces or changes between each reference voltage level, each comparator will output either a HIGH or a LOW condition to the encoder which in turn produces a 2-bit binary code between 00 and 11 relative to V_{IN} .

This is all well and good, but priority encoders are not available as 4-to-2 bit devices, and if we use a commercially available one such as the TTL 74LS148 or its CMOS 4532 equivalent which are both 8-bit devices, then six of the binary bits would not be used. But a simple encoder circuit can be made using digital Ex-OR gates and a matrix of signal diodes as shown.

2-bit ADC Using Diodes



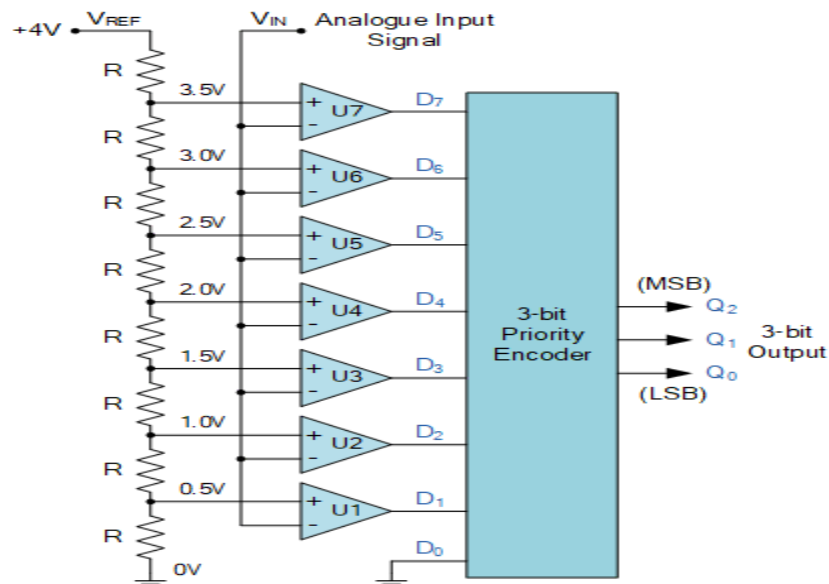
Here the outputs of the comparators are encoded using Exclusive-OR gates before being fed to the diodes. Two external pull-down resistor are used at their outputs and ground (0V) to ensure a LOW condition and stop the outputs from floating when the diodes are reverse biased. So as with the previous circuit, depending upon the value of V_{IN} determines which comparator produces an output signal HIGH (or LOW) to the exclusive-OR gates producing a HIGH output if one input or the other input is HIGH, but not both, (Boolean expression is $Q = A.B + A.B$). These Ex-OR gates could also be constructed using the combinational logic AND–OR–NAND gates.

The problem here with both designs of 4-to-2 converters is that the *resolution* of this simple 2-bit A/D converter is 1 volt because as we have seen, the analogue input voltage at V_{IN} must change by one full volt in order for the encoder to change its output code. One way to improve the resolution of the output is to increase it to a 3-bit A/D converter using more comparators.

3-bit Analogue-to-Digital Converter

The parallel ADC above converts the analogue input voltage in the range from 0 to over 3 volts to produce a 2-bit binary code. Since a 3-bit digital logic system can generate $2^3 = 8$ different digital outputs, the analogue input voltage can therefore be compared against eight reference voltage levels with each voltage level being equal to one eighth ($V/8$) of the reference voltage. Thus we could now measure a resolution of 0.5 ($4/8$) volts and would require $2^3 - 1$ comparators for a 3-bit binary code output between 000 (0) and 111 (7) as shown.

3-bit Analogue to Digital Converter Circuit



This will give us a 3-bit output code for all eight possible values of analogue input of:

3-bit A/D converter Output

[illegible]

Where again “X” is a “don’t care”, that is either a logic “0” or a logic “1” input condition.

Then we can see, that by increasing the resolution of the ADC, not only increases the number of output binary bits, but also increases the number of comparators and voltage levels needed. So a 4-bit resolution requires 15 ($2^4 - 1$) comparators, an 8-bit resolution requires 255 ($2^8 - 1$) comparators, while a 10-bit analogue-to-digital converter would require 1023 comparators, etc. So for this type of **Analogue-to-Digital Converter** circuit, the higher the number of output bits required, the more complex becomes the circuit.

However, the advantage of this type of parallel or flash A/D converter is that its real-time conversion rate is relatively fast and can be easily built as part of a project if only a few binary bits are required to produce a read out on a digital display to show the voltage value of an analogue input signal.

As well as taking an analogue input signal from a sensor or transducer and converting it using an analogue-to-digital converter into a digital binary code as part of an input interfacing circuit, we can also take a binary code and convert it into an equivalent analogue quantity using a Digital-to-Analogue Converter for output interfacing to control a motor or actuator, or commonly in audio applications.

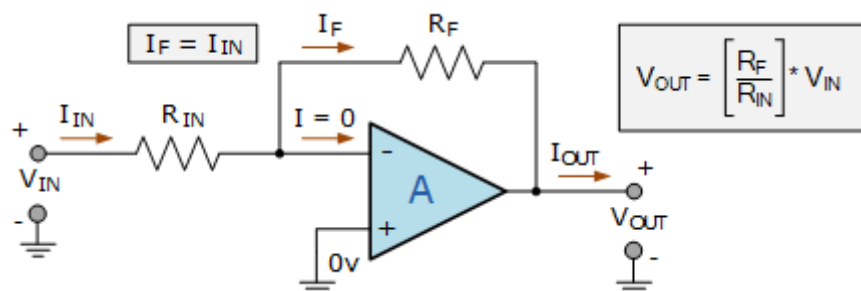
In the next tutorial about digital circuits, we will look at the a **digital-to-analogue converter**, or simply DAC, which are the exact opposite of the *analogue-to-digital converters* looked at here. DAC’s use op-amps and resistive divider networks to convert an “n”-bit binary number into an equivalent analogue output voltage, or current signal.

Digital-to-Analogue Converters, or **DAC’s** as they are more commonly known, are the opposite of the Analogue-to-Digital Converters we looked at in a previous tutorial. DAC’s convert binary or non-binary numbers and codes into analogue ones with its output voltage (or current) being proportional to the value of its digital input number. For example, we may have a 4-bit digital logic circuit that ranges from 0000 to 1111₂, (0 to F₁₆) which a DAC converts to a voltage output ranging from 0 to 10V.

Converting an “n”-bit digital input code into an equivalent analogue output voltage between 0 and some V_{MAX} value can be done in a number of ways, but the most common and easily understood conversion methods uses a weighted resistors and a summing amplifier, or a R-2R resistor ladder network and operational amplifier. Both *digital-to-analogue conversion* methods produce a weighted sum output, with the weights set by the resistive values used in the ladder networks contributing a different “weighted” amount to the signals output.

We saw in our tutorial section about **Operational Amplifiers** that an inverting amplifier uses negative feedback to reduce its open-loop gain, A_{OL} and does so by feeding back a fraction of its output signal back to the input. We also saw that the input voltage V_{IN} is connected directly to its inverting input via a resistor R_{IN} and that the inverting amplifiers closed-loop voltage gain, $A_{V(CL)}$ is determined by the ratio of these two resistors as shown.

Inverting Operational Amplifier Circuit



Then we can see that V_{OUT} is given as V_{IN} multiplied by the closed-loop Gain (A_{CL}), which is determined by the ratio of the feedback resistance, R_F to the input resistance, R_{IN} . So by altering the values of either R_F or R_{IN} we can change the closed-loop gain of the op-amp and therefore the value of V_{OUT} ($I_F * R_i$) for a given input signal. Here in this inverting operational amplifier example we have used a single input voltage signal, but what if we added another input resistor to combine two or more analogue signals into a single output, what would be the effect on the circuit and its gain.

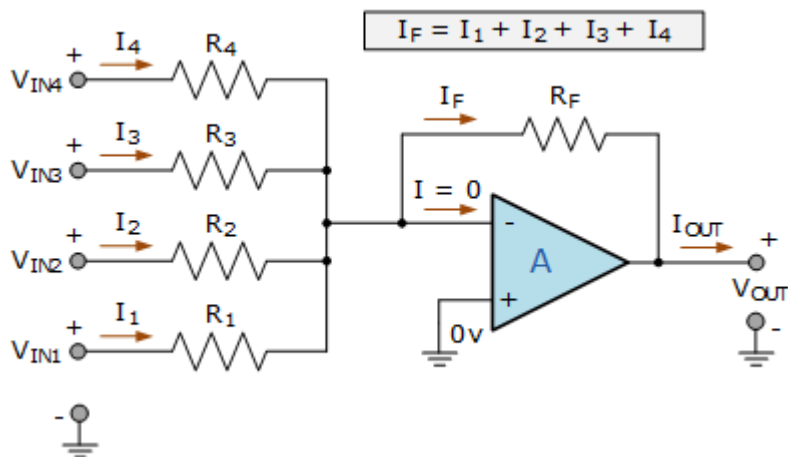
Digital-to-Analogue Converter Summing Amplifier

By connecting multiple inputs to the negative terminal of the operational amplifier, we can convert the single input circuit from above into a *summing amplifier* or to be more precise, a “summing inverting voltage amplifier” circuit.

As the negative feedback created by the feedback resistor, R_F biases the inverting input of the op-amp at zero potential, any input signals are effectively electrically isolated from each other with the output being the inverted sum of all the input signals combined. Thus a summing amplifier in the inverting mode produces the negative sum

of any number of input voltages, whereas a no-inverting summing amplifier would produce the positive sum of any number of input voltages. Consider the circuit below.

Inverting Summing Amplifier Circuit



In the summing amplifier circuit above, the output voltage, (V_{OUT}) is proportional to the sum of the four input voltages, V_{IN1} , V_{IN2} , V_{IN3} , and V_{IN4} and we can modify the original equation for the inverting amplifier configuration above to take account of these four new input values as follows:

$$I_F = I_1 + I_2 + I_3 + I_4 = \frac{V_{IN1}}{R_1} + \frac{V_{IN2}}{R_2} + \frac{V_{IN3}}{R_3} + \frac{V_{IN4}}{R_4}$$

$$V_{OUT} = \frac{R_F}{I_F} \times V_{IN} = - \left(\frac{R_F}{R_1} V_{IN1} + \frac{R_F}{R_2} V_{IN2} + \frac{R_F}{R_3} V_{IN3} + \frac{R_F}{R_4} V_{IN4} \right)$$

$$\therefore V_{OUT} = - \frac{R_F}{R_{IN}} (V_{IN1} + V_{IN2} + V_{IN3} + V_{IN4})$$

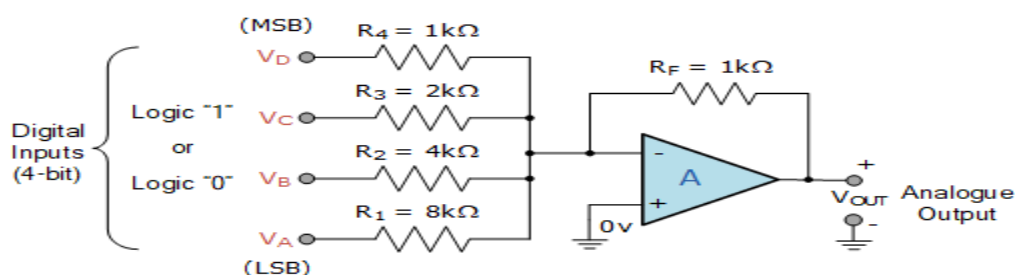
Then we can see that the output voltage is an inverted, scaled sum of the four input voltages as each input voltage is multiplied by its corresponding gain and added to the next to produce the total output. If all the resistances are the same and of an equal value, that is: $R_F = R_1 = R_2 = R_3 = R_4$, then each input channel will have a closed-loop voltage gain of unity (1) so the output voltage is given simply by:

$$V_{OUT} = -(V_{IN1} + V_{IN2} + V_{IN3} + V_{IN4})$$

If we now assume that the four inputs of the summing amplifier are binary inputs with voltage values of either 0 or 5 volts (LOW or HIGH, 0 or 1) and we double the resistive values of each input resistor with regards to the previous one, we can produce an output condition which would be the weighted sum of these four input voltages creating the basic circuit for a 4-bit binary weighted digital-to-analogue converter, or 4-bit weighted D/A converter.

Labelling the four summing inputs as A, B, C, D and making $R_F = 1k\Omega$, with the four input resistors ranging from $1k\Omega$ to $8k\Omega$ (or multiples thereof), we can construct a simple 4-bit binary weighted analogue-to-digital converter circuit as shown.

4-bit Binary Weighted Digital-to-Analogue Converter



For a 4-bit binary number there are $2^4 = 16$ possible combinations or A, B, C, and D ranging from 0000_2 to 1111_2 which corresponds to decimal 0 to 15 respectively. If we make the weight of each input bit double with respect to the other, we end up with an 8-4-2-1 binary code ratio corresponding to 2^3 , 2^2 , 2^1 and 2^0 .

So if we set the “D” input resistance at $1\text{k}\Omega$, the “C” input resistance at $2\text{k}\Omega$ (that is the double of D), the “B” input resistance at $4\text{k}\Omega$ (double C), and the “A” input resistance at $8\text{k}\Omega$ (double B), with the feedback resistance R_F set again at $1\text{k}\Omega$, then the transfer characteristic of the 4-bit binary weighted digital-to-analogue converter would be:

4-bit DAC Transfer Characteristic

$$V_{\text{OUT}} = - \left[\frac{R_F}{R_4} V_D + \frac{R_F}{R_3} V_C + \frac{R_F}{R_2} V_B + \frac{R_F}{R_1} V_A \right]$$

$$V_{\text{OUT}} = - \left[\frac{1\text{k}\Omega}{1\text{k}\Omega} V_D + \frac{1\text{k}\Omega}{2\text{k}\Omega} V_C + \frac{1\text{k}\Omega}{4\text{k}\Omega} V_B + \frac{1\text{k}\Omega}{8\text{k}\Omega} V_A \right]$$

$$V_{\text{OUT}} = - \left[1V_D + \frac{1}{2} V_C + \frac{1}{4} V_B + \frac{1}{8} V_A \right]$$

So we can see that if a TTL voltage of +5 volts (logic 1) is applied to the summing amplifiers input, V_D which represents the most significant bit (MSB), the op-amp’s gain will be $R_F/R_4 = 1\text{k}\Omega/1\text{k}\Omega = 1$ (unity). Thus with a 4-bit binary code of 1000 applied, the output of the digital-to-analogue converter circuit will be -5 volts. Likewise, if +5 volts (logic 1) is applied to the summing amplifiers input V_C , the op-amp’s gain will be $R_F/R_3 = 1\text{k}\Omega/2\text{k}\Omega = 1/2$ (one half). So the 4-bit binary code of 0100 would produce an analogue output voltage of -2.5 volts.

Again with a logic “1” applied to the summing amplifiers input V_B , the op-amp’s gain will be $R_F/R_2 = 1\text{k}\Omega/4\text{k}\Omega = 1/4$ (one quarter) with the 4-bit binary code of 0010 producing an output voltage of -1.25 volts, and finally a logic “1” applied to the summing amplifiers input, V_A which represents the least significant bit (LSB), the op-amp’s gain will therefore be $R_F/R_1 = 1\text{k}\Omega/8\text{k}\Omega = 1/8$ (one eighth) with the 4-bit binary code of 0001 producing an output voltage of -0.625 volts, (a 12.5% resolution).

The resolution of this simple 8-4-2-1 binary weighted digital-to-analogue converter will produce an output voltage change of 0.625 volts per 1-bit change in the binary number, and we can express this output voltage change in the following table.

4-bit Binary Weighted D/A Converter Output

Digital Inputs				V_{OUT} Expression	V_{OUT}
D	C	B	A	$1*V_D + 1/2*V_C + 1/4*V_B + 1/8*V_A$	in Volts
0	0	0	0	$0*5 + 0*5 + 0*5 + 0*5$	0
0	0	0	1	$0*5 + 0*5 + 0*5 + 1/8*5$	-0.625
0	0	1	0	$0*5 + 0*5 + 1/4*5 + 0*5$	-1.25
0	0	1	1	$0*5 + 0*5 + 1/4*5 + 1/8*5$	-1.875

0	1	0	0	$0*5 + \frac{1}{2}*5 + 0*5 + 0*5$	-2.50
0	1	0	1	$0*5 + \frac{1}{2}*5 + 0*5 + \frac{1}{8}*5$	-3.125
0	1	1	0	$0*5 + \frac{1}{2}*5 + \frac{1}{4}*5 + 0*5$	-3.75
0	1	1	1	$0*5 + \frac{1}{2}*5 + \frac{1}{4}*5 + \frac{1}{8}*5$	-4.375
1	0	0	0	$1*5 + 0*5 + 0*5 + 0*5$	-5.00
1	0	0	1	$1*5 + 0*5 + 0*5 + \frac{1}{8}*5$	-5.625
1	0	1	0	$1*5 + 0*5 + \frac{1}{4}*5 + 0*5$	-6.25
1	0	1	1	$1*5 + 0*5 + \frac{1}{4}*5 + \frac{1}{8}*5$	-6.875
1	1	0	0	$1*5 + \frac{1}{2}*5 + 0*5 + 0*5$	-7.50
1	1	0	1	$1*5 + \frac{1}{2}*5 + 0*5 + \frac{1}{8}*5$	-8.125
1	1	1	0	$1*5 + \frac{1}{2}*5 + \frac{1}{4}*5 + 0*5$	-8.75
1	1	1	1	$1*5 + \frac{1}{2}*5 + \frac{1}{4}*5 + \frac{1}{8}*5$	-9.375

Where the output voltages are all negative due to the inverting input of the summing amplifier.

By increasing the number of binary digits and the resistive summing network so that each resistor has a different weighting, the resolution of the analogue output voltage for a binary weighted digital-to-analogue converter can be increased. For example, an 8-bit DAC with TTL +5 inputs would produce a resolution of 0.039 ($1/128*V$) volts, while a 12-bit DAC would be 0.00244 ($1/2048*V$) volts per step (1 LSB) change of the input binary (or non-binary) code.

Clearly then the disadvantage here is that a binary weighted resistor DAC requires a large range of high precision resistors (one per bit) for an “n”-bit DAC making it impractical (and expensive) for converters with more than a just a few bits of resolution. But we can expand on this idea of a binary weighted digital-to-analogue circuit configuration which uses different value resistors one step further by converting it into a R-2R resistor ladder DAC which requires only two precision resistance values, namely R and 2R.

In the next tutorial about **Digital-to-Analogue Converters**, we will look at how the *R-2R Digital-to-Analogue Converter* uses just two resistor values to convert a digital binary number into an analogue voltage output.